

Project 1: Supervised Learning

T504 Machine Learning / T711 Fundamentals of Machine Learning

Fall, 2026

1 Introduction

The goal of this project is to apply your theoretical knowledge about supervised learning algorithms and the workflow to solve a classification problem in practice. In particular, the task is to use a realistic dataset for a **supervised learning task**, design the process of preprocessing the data, selecting a model and its hyperparameters, and implement a program that automates this process. You need to hand in a report outlining the process, justifying all design decisions, and presenting the results.

This project can be done in groups of up to 3 students. (The groups do not have to be the same as for the labs.) The intended workload for this project is about 15-20 hours per student (not counting computation time, that is, assuming you set up the program, let it run and do something else while you wait for the results). Start working on this project in time. There will be some waiting for results of your program.

2 Data Set

You need to select the problem you want to solve, that is, which data set to learn from. The data needs to be labeled (that is, have a defined output) such that it is suitable for supervised learning and should have a decent size (at least some thousand instances). Be aware that some available data sets require additional pre-processing before they can be used for training a model.

You can find datasets to use online, e.g., on these sites:

- <https://www.kaggle.com/datasets>
- <https://archive.ics.uci.edu/>

You can also use data from other sources that you have access to.

Advice: After an initial understanding of the data, try some simple model and see if the data set is suitable for the project. What you want to avoid is doing EDA and writing about it and only then start modeling and realizing the data set is not suitable for the task you planned to perform.

Note: If you prefer me providing one suggestion, you could have a look at the "Colon data set" (<https://jundong1.github.io/scikit-feature/datasets>).

html) or another "Biological Data" data set found on this Github repository. It has many more features than observations, making it an interesting challenge to avoid overfitting and attempt some dimensionality reduction. Here also using some k-fold cross-validation instead of a simple holdout split may be good to be able to provide e.g., an average accuracy, as having few observations in a test set makes the result less robust and more affected by chance variation (in the sampling).

3 Tasks

1. Design a process to find a good model for the data set. This includes deciding
 - How to split the data into training / test set?
 - What is the benchmark you will compare your model with?
 - Which models have a good chance of working well on this data set?
 - What preprocessing needs to be done to use these models?
 - How to set the hyperparameters of these models and/or which range of values for the hyperparameters should be tried?
 - How to evaluate the different models to decide on the best one?

Justify each one of your decisions! (E.g., if you decide on a set of potential models / values for hyperparameters then say why.) Don't forget to set data aside for testing or you won't be able to report on how well the best model you found actually performs in the end. I suggest for this part to look at publications (e.g., scientific papers) on the respective data set or on similar problems to see which models with which parameters experts used and also how they designed their experiments.

Don't forget to make a rough estimate how much time it will take to execute that process for deciding how many different configurations you can test. Maybe, you will need to do some trials to see how long it takes to train some of the models for the given data set.

2. Automate the process you designed. That is, implement a program or a collection of programs that go through the process of splitting the data, pre-processing it, training and validating different models with different parameter settings to find the best model. The program should print out results for each of the trials and report on the performance of the best model.
3. Write a report in the style of a research paper on your findings. The report should be roughly structured into:
 - Introduction: describing the problem and the data. Explain what you are trying to accomplish (and what is the status quo and/or benchmark for this problem).
 - Process: describing the process for finding the best model and justifying all decision. If you used other papers/sources for your decisions you need to reference them here.

- Results: report on the performance for the different models and hyperparameters. Essentially, take the numbers you got as output of your program and put them into a nice form (plots, tables, ...) that makes them easier to interpret and shows, for example, how the choice of values for different hyperparameters influences the performance.
- Conclusions: Interpret the results and compare with results in the literature (if possible). What can be said about the performance of different models on the problem? How important is the choice of the hyperparameters for the performance of the different models?
- Future Work: Suggest how your results could be improved. What did you learn from the results you got and what would you do differently now that you have learned this?